

```
2 Integer target = getTarget(array1, array2);
3 if (target == null) return null;
4
5 for (int one : array1) {
6     for (int two : array2) {
7         if (one - two == target) {
8             int[] values = {one, two};
9             return values;
10        }
11    }
12 }
13
14 return null;
15 }
16
17 Integer getTarget(int[] array1, int[] array2) {
18     int sum1 = sum(array1);
19     int sum2 = sum(array2);
20
21     if ((sum1 - sum2) % 2 != 0) return null;
22     return (sum1 - sum2) / 2;
23 }
```

We've used an `Integer` (a boxed data type) as the return value for `getTarget`. This allows us to distinguish an "error" case.

This algorithm takes $O(AB)$ time.

Optimal Solution

This problem reduces to finding a pair of values that have a particular difference. With that in mind, let's revisit what the brute force does.

In the brute force, we're looping through *A* and then, for each element, looking for an element in *B* which gives us the "right" difference. If the value in *A* is 5 and the target is 3, then we must be looking for the value 2. That's the only value that could fulfill the goal.

That is, rather than writing `one - two == target`, we could have written `two == one - target`. How can we more quickly find an element in *B* that equals `one - target`?

We can do this very quickly with a hash table. We just throw all the elements in *B* into a hash table. Then, iterate through *A* and look for the appropriate element in *B*.

```
1 int[] findSwapValues(int[] array1, int[] array2) {
2     Integer target = getTarget(array1, array2);
3     if (target == null) return null;
4     return findDifference(array1, array2, target);
5 }
6
7 /* Find a pair of values with a specific difference. */
8 int[] findDifference(int[] array1, int[] array2, int target) {
9     HashSet<Integer> contents2 = getContents(array2);
10    for (int one : array1) {
11        int two = one - target;
12        if (contents2.contains(two)) {
13            int[] values = {one, two};
14            return values;
15        }
16    }
17 }
```

```

16     }
17
18     return null;
19 }
20
21 /* Put contents of array into hash set. */
22 HashSet<Integer> getContents(int[] array) {
23     HashSet<Integer> set = new HashSet<Integer>();
24     for (int a : array) {
25         set.add(a);
26     }
27     return set;
28 }

```

This solution will take $O(A+B)$ time. This is the Best Conceivable Runtime (BCR), since we have to at least touch every element in the two arrays.

Alternate Solution

If the arrays are sorted, we can iterate through them to find an appropriate pair. This will require less space.

```

1  int[] findSwapValues(int[] array1, int[] array2) {
2      Integer target = getTarget(array1, array2);
3      if (target == null) return null;
4      return findDifference(array1, array2, target);
5  }
6
7  int[] findDifference(int[] array1, int[] array2, int target) {
8      int a = 0;
9      int b = 0;
10
11     while (a < array1.length && b < array2.length) {
12         int difference = array1[a] - array2[b];
13         /* Compare difference to target. If difference is too small, then make it
14          * bigger by moving a to a bigger value. If it is too big, then make it
15          * smaller by moving b to a bigger value. If it's just right, return this
16          * pair. */
17         if (difference == target) {
18             int[] values = {array1[a], array2[b]};
19             return values;
20         } else if (difference < target) {
21             a++;
22         } else {
23             b++;
24         }
25     }
26
27     return null;
28 }

```

This algorithm takes $O(A + B)$ time but requires the arrays to be sorted. If the arrays aren't sorted, we can still apply this algorithm but we'd have to sort the arrays first. The overall runtime would be $O(A \log A + B \log B)$.

16.22 Langton's Ant: An ant is sitting on an infinite grid of white and black squares. It initially faces right.

At each step, it does the following:

- (1) At a white square, flip the color of the square, turn 90 degrees right (clockwise), and move forward one unit.
- (2) At a black square, flip the color of the square, turn 90 degrees left (counter-clockwise), and move forward one unit.

Write a program to simulate the first *K* moves that the ant makes and print the final board as a grid. Note that you are not provided with the data structure to represent the grid. This is something you must design yourself. The only input to your method is *K*. You should print the final grid and return nothing. The method signature might be something like `void printKMoves(int K)`.

pg 185

SOLUTION

At first glance, this problem seems very straightforward: create a grid, remember the ant's position and orientation, flip the cells, turn, and move. The interesting part comes in how to handle an infinite grid.

Solution #1: Fixed Array

Technically, since we're only running the first *K* moves, we do have a max size for the grid. The ant cannot move more than *K* moves in either direction. If we create a grid that has width *2K* and height *2K* (and place the ant at the center), we know it will be big enough.

The problem with this is that it's not very extensible. If you run *K* moves and then want to run another *K* moves, you might be out of luck.

Additionally, this solution wastes a good amount of space. The max might be *K* moves in a particular dimension, but the ant is probably going in circles a bit. You probably won't need all this space.

Solution #2: Resizable Array

One thought is to use a resizable array, such as Java's `ArrayList` class. This allows us to grow an array as necessary, while still offering $O(1)$ amortized insertion.

The problem is that our grid needs to grow in two dimensions, but the `ArrayList` is only a single array. Additionally, we need to grow "backward" into negative values. The `ArrayList` class doesn't support this.

However, we take a similar approach by building our own resizable grid. Each time the ant hits an edge, we double the size of the grid in that dimension.

What about the negative expansions? While conceptually we can talk about something being at negative positions, we cannot actually access array indices with negative values.

One way we can handle this is to create "fake indices." Let us treat the ant as being at coordinates $(-3, -10)$, but track some sort of offset or delta to translate these coordinates into array indices.

This is actually unnecessary, though. The ant's location does not need to be publicly exposed or consistent (unless, of course, indicated by the interviewer). When the ant travels into negative coordinates, we can double the size of the array and just move the ant and all cells into the positive coordinates. Essentially, we are relabeling all the indices.

This relabeling will not impact the big *O* time since we have to create a new matrix anyway.

```
1 public class Grid {  
2     private boolean[][] grid;
```

```

3     private Ant ant = new Ant();
4
5     public Grid() {
6         grid = new boolean[1][1];
7     }
8
9     /* Copy old values into new array, with an offset/shift applied to the row and
10    * columns. */
11    private void copyWithShift(boolean[][] oldGrid, boolean[][] newGrid,
12                               int shiftRow, int shiftColumn) {
13        for (int r = 0; r < oldGrid.length; r++) {
14            for (int c = 0; c < oldGrid[0].length; c++) {
15                newGrid[r + shiftRow][c + shiftColumn] = oldGrid[r][c];
16            }
17        }
18    }
19
20    /* Ensure that the given position will fit on the array. If necessary, double
21    * the size of the matrix, copy the old values over, and adjust the ant's
22    * position so that it's in a positive range. */
23    private void ensureFit(Position position) {
24        int shiftRow = 0;
25        int shiftColumn = 0;
26
27        /* Calculate new number of rows. */
28        int numRows = grid.length;
29        if (position.row < 0) {
30            shiftRow = numRows;
31            numRows *= 2;
32        } else if (position.row >= numRows) {
33            numRows *= 2;
34        }
35
36        /* Calculate new number of columns. */
37        int numColumns = grid[0].length;
38        if (position.column < 0) {
39            shiftColumn = numColumns;
40            numColumns *= 2;
41        } else if (position.column >= numColumns) {
42            numColumns *= 2;
43        }
44
45        /* Grow array, if necessary. Shift ant's position too. */
46        if (numRows != grid.length || numColumns != grid[0].length) {
47            boolean[][] newGrid = new boolean[numRows][numColumns];
48            copyWithShift(grid, newGrid, shiftRow, shiftColumn);
49            ant.adjustPosition(shiftRow, shiftColumn);
50            grid = newGrid;
51        }
52    }
53
54    /* Flip color of cells. */
55    private void flip(Position position) {
56        int row = position.row;
57        int column = position.column;
58        grid[row][column] = grid[row][column] ? false : true;

```

```

59     }
60
61     /* Move ant. */
62     public void move() {
63         ant.turn(grid[ant.position.row][ant.position.column]);
64         flip(ant.position);
65         ant.move();
66         ensureFit(ant.position); // grow
67     }
68
69     /* Print board. */
70     public String toString() {
71         StringBuilder sb = new StringBuilder();
72         for (int r = 0; r < grid.length; r++) {
73             for (int c = 0; c < grid[0].length; c++) {
74                 if (r == ant.position.row && c == ant.position.column) {
75                     sb.append(ant.orientation);
76                 } else if (grid[r][c]) {
77                     sb.append("X");
78                 } else {
79                     sb.append("_");
80                 }
81             }
82             sb.append("\n");
83         }
84         sb.append("Ant: " + ant.orientation + ". \n");
85         return sb.toString();
86     }
87 }

```

We pulled the Ant code into a separate class. The nice thing about this is that if we need to have multiple ants for some reason, we can easily extend the code to support this.

```

1  public class Ant {
2      public Position position = new Position(0, 0);
3      public Orientation orientation = Orientation.right;
4
5      public void turn(boolean clockwise) {
6          orientation = orientation.getTurn(clockwise);
7      }
8
9      public void move() {
10         if (orientation == Orientation.left) {
11             position.column--;
12         } else if (orientation == Orientation.right) {
13             position.column++;
14         } else if (orientation == Orientation.up) {
15             position.row--;
16         } else if (orientation == Orientation.down) {
17             position.row++;
18         }
19     }
20
21     public void adjustPosition(int shiftRow, int shiftColumn) {
22         position.row += shiftRow;
23         position.column += shiftColumn;
24     }
25 }

```

Orientation is also its own enum, with a few useful functions.

```

1  public enum Orientation {
2      left, up, right, down;
3
4      public Orientation getTurn(boolean clockwise) {
5          if (this == left) {
6              return clockwise ? up : down;
7          } else if (this == up) {
8              return clockwise ? right : left;
9          } else if (this == right) {
10             return clockwise ? down : up;
11          } else { // down
12              return clockwise ? left : right;
13          }
14      }
15
16      @Override
17      public String toString() {
18          if (this == left) {
19              return "\u2190";
20          } else if (this == up) {
21              return "\u2191";
22          } else if (this == right) {
23              return "\u2192";
24          } else { // down
25              return "\u2193";
26          }
27      }
28  }

```

We've also put `Position` into its own simple class. We could just as easily track the row and column separately.

```

1  public class Position {
2      public int row;
3      public int column;
4
5      public Position(int row, int column) {
6          this.row = row;
7          this.column = column;
8      }
9  }

```

This works, but it's actually more complicated than is necessary.

Solution #3: HashSet

Although it may seem "obvious" that we would use a matrix to represent a grid, it's actually easier not to do that. All we actually need is a list of the white squares (as well as the ant's location and orientation).

We can do this by using a `HashSet` of the white squares. If a position is in the hash set, then the square is white. Otherwise, it is black.

The one tricky bit is how to print the board. Where do we start printing? Where do we end?

Since we will need to print a grid, we can track what should be top-left and bottom-right corner of the grid. Each time the ant moves, we compare the ant's position to the most top-left position and most bottom-right position, updating them if necessary.

```
1 public class Board {
2     private HashSet<Position> whites = new HashSet<Position>();
3     private Ant ant = new Ant();
4     private Position topLeftCorner = new Position(0, 0);
5     private Position bottomRightCorner = new Position(0, 0);
6
7     public Board() { }
8
9     /* Move ant. */
10    public void move() {
11        ant.turn(isWhite(ant.position)); // Turn
12        flip(ant.position); // flip
13        ant.move(); // move
14        ensureFit(ant.position);
15    }
16
17    /* Flip color of cells. */
18    private void flip(Position position) {
19        if (whites.contains(position)) {
20            whites.remove(position);
21        } else {
22            whites.add(position.clone());
23        }
24    }
25
26    /* Grow grid by tracking the most top-left and bottom-right positions.*/
27    private void ensureFit(Position position) {
28        int row = position.row;
29        int column = position.column;
30
31        topLeftCorner.row = Math.min(topLeftCorner.row, row);
32        topLeftCorner.column = Math.min(topLeftCorner.column, column);
33
34        bottomRightCorner.row = Math.max(bottomRightCorner.row, row);
35        bottomRightCorner.column = Math.max(bottomRightCorner.column, column);
36    }
37
38    /* Check if cell is white. */
39    public boolean isWhite(Position p) {
40        return whites.contains(p);
41    }
42
43    /* Check if cell is white. */
44    public boolean isWhite(int row, int column) {
45        return whites.contains(new Position(row, column));
46    }
47
48    /* Print board. */
49    public String toString() {
50        StringBuilder sb = new StringBuilder();
51        int rowMin = topLeftCorner.row;
52        int rowMax = bottomRightCorner.row;
53        int colMin = topLeftCorner.column;
54        int colMax = bottomRightCorner.column;
55        for (int r = rowMin; r <= rowMax; r++) {
56            for (int c = colMin; c <= colMax; c++) {
```

```

57         if (r == ant.position.row && c == ant.position.column) {
58             sb.append(ant.orientation);
59         } else if (isWhite(r, c)) {
60             sb.append("X");
61         } else {
62             sb.append("_");
63         }
64     }
65     sb.append("\n");
66 }
67 sb.append("Ant: " + ant.orientation + ". \n");
68 return sb.toString();
69 }

```

The implementation of `Ant` and `Orientation` is the same.

The implementation of `Position` gets updated slightly, in order to support the `HashSet` functionality. The position will be the key, so we need to implement a `hashCode()` function.

```

1  public class Position {
2      public int row;
3      public int column;
4
5      public Position(int row, int column) {
6          this.row = row;
7          this.column = column;
8      }
9
10     @Override
11     public boolean equals(Object o) {
12         if (o instanceof Position) {
13             Position p = (Position) o;
14             return p.row == row && p.column == column;
15         }
16         return false;
17     }
18
19     @Override
20     public int hashCode() {
21         /* There are many options for hash functions. This is one. */
22         return (row * 31) ^ column;
23     }
24
25     public Position clone() {
26         return new Position(row, column);
27     }
28 }

```

The nice thing about this implementation is that if we do need to access a particular cell elsewhere, we have consistent row and column labeling.

16.23 Rand7 from Rand5: Implement a method `rand7()` given `rand5()`. That is, given a method that generates a random number between 0 and 4 (inclusive), write a method that generates a random number between 0 and 6 (inclusive).

pg 186

SOLUTION

To implement this function correctly, we must have each of the values between 0 and 6 returned with 1/7th probability.

First Attempt (Fixed Number of Calls)

As a first attempt, we might try generating all numbers between 0 and 9, and then mod the resulting value by 7. Our code for it might look something like this:

```
1 int rand7() {
2     int v = rand5() + rand5();
3     return v % 7;
4 }
```

Unfortunately, the above code will not generate the values with equal probability. We can see this by looking at the results of each call to `rand5()` and the return result of the `rand7()` function.

1st Call	2nd Call	Result	1st Call	2nd Call	Result
0	0	0	2	3	5
0	1	1	2	4	6
0	2	2	3	0	3
0	3	3	3	1	4
0	4	4	3	2	5
1	0	1	3	3	6
1	1	2	3	4	0
1	2	3	4	0	4
1	3	4	4	1	5
1	4	5	4	2	6
2	0	2	4	3	0
2	1	3	4	4	1
2	2	4			

Each individual row has a 1 in 25 chance of occurring, since there are two calls to `rand5()` and each distributes its results with $\frac{1}{5}$ th probability. If you count up the number of times each number occurs, you'll note that this `rand7()` function will return 4 with $\frac{5}{25}$ th probability but return 0 with just $\frac{3}{25}$ th probability. This means that our function has failed; the results do not have probability $\frac{1}{7}$ th.

Now, imagine we modify our function to add an if-statement, to change the constant multiplier, or to insert a new call to `rand5()`. We will still wind up with a similar looking table, and the probability of getting any one of those rows will be $\frac{1}{5^k}$, where k is the number of calls to `rand5()` in that row. Different rows may have different number of calls.

The probability of winding up with the result of the `rand7()` function being, say, 6 would be the sum of the probabilities of all rows that result in 6. That is:

$$P(\text{rand7}() = 6) = \frac{1}{5^1} + \frac{1}{5^1} + \dots + \frac{1}{5^m}$$

We know that, in order for our function to be correct, this probability must equal $\frac{1}{7}$. This is impossible though. Because 5 and 7 are relatively prime, no series of reciprocal powers of 5 will result in $\frac{1}{7}$.

Does this mean the problem is impossible? Not exactly. Strictly speaking, it means that, as long as we can list out the combinations of `rand5()` results that will result in a particular value of `rand7()`, the function will not give well distributed results.

We can still solve this problem. We just have to use a while loop, and realize that there's no telling just how many turns will be required to return a result.

Second Attempt (Nondeterministic Number of Calls)

As soon as we've allowed for a while loop, our work gets much easier. We just need to generate a range of values where each value is equally likely (and where the range has at least seven elements). If we can do this, then we can discard the elements greater than the previous multiple of 7, and mod the rest of them by 7. This will get us a value within the range of 0 to 6, with each value being equally likely.

In the below code, we generate the range 0 through 24 by doing `5 * rand5() + rand5()`. Then, we discard the values between 21 and 24, since they would otherwise make `rand7()` unfairly weighted towards 0 through 3. Finally, we mod by 7 to give us the values in the range 0 to 6 with equal probability.

Note that because we discard values in this approach, we have no guarantee on the number of `rand5()` calls it may take to return a value. This is what is meant by a *nondeterministic* number of calls.

```
1  int rand7() {
2      while (true) {
3          int num = 5 * rand5() + rand5();
4          if (num < 21) {
5              return num % 7;
6          }
7      }
8  }
```

Observe that doing `5 * rand5() + rand5()` gives us exactly one way of getting each number in its range (0 to 24). This ensures that each value is equally probable.

Could we instead do `2 * rand5() + rand5()`? No, because the values wouldn't be equally distributed. For example, there would be three ways of getting a 6 ($6 = 2 * 1 + 4$, $6 = 2 * 2 + 2$, and $6 = 2 * 3 + 0$) but only one way of getting a 0 ($0 = 2 * 0 + 0$). The values in the range are not equally probable.

There is a way that we can use `2 * rand5()` and still get an identically distributed range, but it's much more complicated. See below.

```
1  int rand7() {
2      while (true) {
3          int r1 = 2 * rand5(); /*evens between 0 and 9 */
4          int r2 = rand5(); /*used later to generate a 0 or 1 */
5          if (r2 != 4) { /*r2 has extra even num-discard the extra */
6              int rand1 = r2 % 2; /*Generate 0 or 1 */
7              int num = r1 + rand1; /*will be in the range 0 to 9 */
8              if (num < 7) {
9                  return num;
10             }
11         }
12     }
13 }
```

In fact, there is an infinite number of ranges we can use. The key is to make sure that the range is big enough and that all values are equally likely.

16.24 Pairs with Sum: Design an algorithm to find all pairs of integers within an array which sum to a specified value.

pg 185

SOLUTION

Let's start with a definition. If we're trying to find a pair of numbers that sums to z , the *complement* of x will be $z - x$ (that is, the number that can be added to x to make z). For example, if we're trying to find a pair of numbers that sums to 12, the complement of -5 would be 17.

Brute Force

A brute force solution is to just iterate through all pairs and print the pair if its sum matches the target sum.

```
1 ArrayList<Pair> printPairSums(int[] array, int sum) {
2     ArrayList<Pair> result = new ArrayList<Pair>();
3     for (int i = 0 ; i < array.length; i++) {
4         for (int j = i + 1; j < array.length; j++) {
5             if (array[i] + array[j] == sum) {
6                 result.add(new Pair(array[i], array[j]));
7             }
8         }
9     }
10    return result;
11 }
```

If there are duplicates in the array (e.g., {5, 6, 5}), it might print the same sum twice. You should discuss this with your interviewer.

Optimized Solution

We can optimize this with a hash map, where the value in the hash map reflects the number of “unpaired” instances of a key. We walk through the array. At each element x , check how many unpaired instances of x 's complement preceded it in the array. If the count is at least one, then there is an unpaired instance of x 's complement. We add this pair and decrement x 's complement to signify that this element has been paired. If the count is zero, then increment the value of x in the hash table to signify that x is unpaired.

```
1 ArrayList<Pair> printPairSums(int[] array, int sum) {
2     ArrayList<Pair> result = new ArrayList<Pair>();
3     HashMap<Integer, Integer> unpairedCount = new HashMap<Integer, Integer>();
4     for (int x : array) {
5         int complement = sum - x;
6         if (unpairedCount.containsKey(complement) && unpairedCount.get(complement) > 0) {
7             result.add(new Pair(x, complement));
8             adjustCounterBy(unpairedCount, complement, -1); // decrement complement
9         } else {
10            adjustCounterBy(unpairedCount, x, 1); // increment count
11        }
12    }
13    return result;
14 }
15
```

```

16 void adjustCounterBy(HashMap<Integer, Integer> counter, int key, int delta) {
17     counter.put(key, counter.getOrDefault(key, 0) + delta);
18 }

```

This solution will print duplicate pairs, but will not reuse the same instance of an element. It will take $O(N)$ time and $O(N)$ space.

Alternate Solution

Alternatively, we can sort the array and then find the pairs in a single pass. Consider this array:

`{-2, -1, 0, 3, 5, 6, 7, 9, 13, 14}.`

Let `first` point to the head of the array and `last` point to the end of the array. To find the complement of `first`, we just move `last` backwards until we find it. If `first + last < sum`, then there is no complement for `first`. We can therefore move `first` forward. We stop when `first` is greater than `last`.

Why must this find all complements for `first`? Because the array is sorted and we're trying progressively smaller numbers. When the sum of `first` and `last` is less than the sum, we know that trying even smaller numbers (as `last`) won't help us find a complement.

Why must this find all complements for `last`? Because all pairs must be made up of a `first` and a `last`. We've found all complements for `first`, therefore we've found all complements of `last`.

```

1 void printPairSums(int[] array, int sum) {
2     Arrays.sort(array);
3     int first = 0;
4     int last = array.length - 1;
5     while (first < last) {
6         int s = array[first] + array[last];
7         if (s == sum) {
8             System.out.println(array[first] + " " + array[last]);
9             first++;
10            last--;
11        } else {
12            if (s < sum) first++;
13            else last--;
14        }
15    }
16 }

```

This algorithm takes $O(N \log N)$ time to sort and $O(N)$ time to find the pairs.

Note that since the array is presumably unsorted, it would be equally fast in terms of big O to just do a binary search at each element for its complement. This would give us a two-step algorithm, where each step is $O(N \log N)$.

16.25 LRU Cache: Design and build a “least recently used” cache, which evicts the least recently used item. The cache should map from keys to values (allowing you to insert and retrieve a value associated with a particular key) and be initialized with a max size. When it is full, it should evict the least recently used item. You can assume the keys are integers and the values are strings.

pg 185

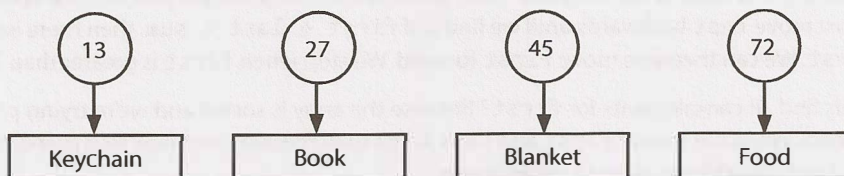
SOLUTION

We should start off by defining the scope of the problem. What exactly do we need to achieve?

- **Inserting Key, Value Pair:** We need to be able to insert a (key, value) pair.

- **Retrieving Value by Key:** We need to be able to retrieve the value using the key.
- **Finding Least Recently Used:** We need to know the least recently used item (and, likely, the usage ordering of all items).
- **Updating Most Recently Used:** When we retrieve a value by key, we need to update the order to be the most recently used item.
- **Eviction:** The cache should have a max capacity and should remove the least recently used item when it hits capacity.

The (key, value) mapping suggests a hash table. This would make it easy to look up the value associated with a particular key.



Unfortunately, a hash table usually would not offer a quick way to remove the most recently used item. We could mark each item with a timestamp and iterate through the hash table to remove the item with the lowest timestamp, but that can get quite slow ($O(N)$ for insertions).

Instead, we could use a linked list, ordered by the most recently used. This would make it easy to mark an item as the most recently used (just put it in the front of the list) or to remove the least recently used item (remove the end).

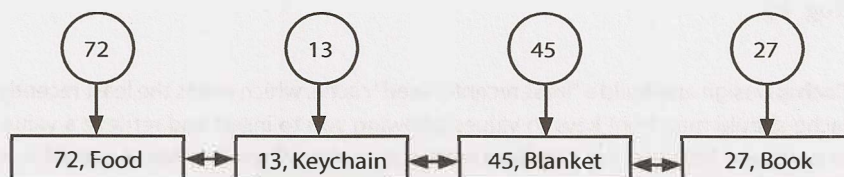


Unfortunately, this does not offer a quick way to look up an item by its key. We could iterate through the linked list and find the item by key. But this could get very slow ($O(N)$ for retrieval).

Each approach does half of the problem (different halves) very well, but neither approach does both parts well.

Can we get the best parts of each? Yes. By using both!

The linked list looks as it did in the earlier example, but now it's a doubly linked list. This allows us to easily remove an element from the middle of the linked list. The hash table now maps to each linked list node rather than the value.



The algorithms now operate as follows:

- **Inserting Key, Value Pair:** Create a linked list node with key, value. Insert into head of linked list. Insert key -> node mapping into hash table.
- **Retrieving Value by Key:** Look up node in hash table and return value. Update most recently used item

(see below).

- **Finding Least Recently Used:** Least recently used item will be found at the end of the linked list.
- **Updating Most Recently Used:** Move node to front of linked list. Hash table does not need to be updated.
- **Eviction:** Remove tail of linked list. Get key from linked list node and remove key from hash table.

The code below implements these classes and algorithms.

```

1  public class Cache {
2      private int maxCacheSize;
3      private HashMap<Integer, LinkedListNode> map =
4          new HashMap<Integer, LinkedListNode>();
5      private LinkedListNode listHead = null;
6      public LinkedListNode listTail = null;
7
8      public Cache(int maxSize) {
9          maxCacheSize = maxSize;
10     }
11
12     /* Get value for key and mark as most recently used. */
13     public String getValue(int key) {
14         LinkedListNode item = map.get(key);
15         if (item == null) return null;
16
17         /* Move to front of list to mark as most recently used. */
18         if (item != listHead) {
19             removeFromLinkedList(item);
20             insertAtFrontOfLinkedList(item);
21         }
22         return item.value;
23     }
24
25     /* Remove node from linked list. */
26     private void removeFromLinkedList(LinkedListNode node) {
27         if (node == null) return;
28
29         if (node.prev != null) node.prev.next = node.next;
30         if (node.next != null) node.next.prev = node.prev;
31         if (node == listTail) listTail = node.prev;
32         if (node == listHead) listHead = node.next;
33     }
34
35     /* Insert node at front of linked list. */
36     private void insertAtFrontOfLinkedList(LinkedListNode node) {
37         if (listHead == null) {
38             listHead = node;
39             listTail = node;
40         } else {
41             listHead.prev = node;
42             node.next = listHead;
43             listHead = node;
44         }
45     }
46
47     /* Remove key/value pair from cache, deleting from hashtable and linked list. */
48     public boolean removeKey(int key) {

```

```

49     LinkedListNode node = map.get(key);
50     removeFromLinkedList(node);
51     map.remove(key);
52     return true;
53 }
54
55 /* Put key, value pair in cache. Removes old value for key if necessary. Inserts
56  * pair into linked list and hash table.*/
57 public void setKeyValue(int key, String value) {
58     /* Remove if already there. */
59     removeKey(key);
60
61     /* If full, remove least recently used item from cache. */
62     if (map.size() >= maxCacheSize && listTail != null) {
63         removeKey(listTail.key);
64     }
65
66     /* Insert new node. */
67     LinkedListNode node = new LinkedListNode(key, value);
68     insertAtFrontOfLinkedList(node);
69     map.put(key, node);
70 }
71
72 private static class LinkedListNode {
73     private LinkedListNode next, prev;
74     public int key;
75     public String value;
76     public LinkedListNode(int k, String v) {
77         key = k;
78         value = v;
79     }
80 }
81 }

```

Note that we've chosen to make `LinkedListNode` an inner class of `Cache`, since no other classes should need access to this class and really should only exist within the scope of `Cache`.

16.26 Calculator: Given an arithmetic equation consisting of positive integers, `+`, `-`, `*` and `/` (no parentheses), compute the result.

EXAMPLE

Input: `2*3+5/6*3+15`

Output: `23.5`

pg 185

SOLUTION

The first thing we should realize is that the dumb thing—just applying each operator left to right—won't work. Multiplication and division are considered "higher priority" operations, which means that they have to happen before addition.

For example, if you have the simple expression `3+6*2`, the multiplication must be performed first, and then the addition. If you just processed the equation left to right, you would end up with the incorrect result, 18, rather than the correct one, 15. You know all of this, of course, but it's worth really spelling out what it means.

Solution #1

We can still process the equation from left to right; we just have to be a little smarter about how we do it. Multiplication and division need to be grouped together such that whenever we see those operations, we perform them immediately on the surrounding terms.

For example, suppose we have this expression:

$$2 - 6 - 7 * 8 / 2 + 5$$

It's fine to compute $2 - 6$ immediately and store it into a `result` variable. But, when we see $7 * (\text{something})$, we know we need to fully process that term before adding it to the result.

We can do this by reading left to right and maintaining two variables.

- The first is processing, which maintains the result of the current cluster of terms (both the operator and the value). In the case of addition and subtraction, the cluster will be just the current term. In the case of multiplication and division, it will be the full sequence (until you get to the next addition or subtraction).
- The second is the `result` variable. If the next term is an addition or subtraction (or there is no next term), then processing is applied to `result`.

On the above example, we would do the following:

1. Read $+2$. Apply it to processing. Apply processing to result. Clear processing.
`processing = {+, 2} --> null`
`result = 0 --> 2`
2. Read -6 . Apply it to processing. Apply processing to result. Clear processing.
`processing = {-, 6} --> null`
`result = 2 --> -4`
3. Read -7 . Apply it to processing. Observe next sign is a $*$. Continue.
`processing = {-, 7}`
`result = -4`
4. Read $*8$. Apply it to processing. Observe next sign is a $/$. Continue.
`processing = {-, 56}`
`result = -4`
5. Read $/2$. Apply it to processing. Observe next sign is a $+$, which terminates this multiplication and division cluster. Apply processing to result. Clear processing.
`processing = {-, 28} --> null`
`result = -4 --> -32`
6. Read $+5$. Apply it to processing. Apply processing to result. Clear processing.
`processing = {+, 5} --> null`
`result = -32 --> -27`

The code below implements this algorithm.

```

1  /* Compute the result of the arithmetic sequence. This works by reading left to
2   * right and applying each term to a result. When we see a multiplication or
3   * division, we instead apply this sequence to a temporary variable. */
4  double compute(String sequence) {
5      ArrayList<Term> terms = Term.parseTermSequence(sequence);
6      if (terms == null) return Integer.MIN_VALUE;
7
8      double result = 0;
9      Term processing = null;
10     for (int i = 0; i < terms.size(); i++) {

```

```

11     Term current = terms.get(i);
12     Term next = i + 1 < terms.size() ? terms.get(i + 1) : null;
13
14     /* Apply the current term to "processing". */
15     processing = collapseTerm(processing, current);
16
17     /* If next term is + or -, then this cluster is done and we should apply
18      * "processing" to "result". */
19     if (next == null || next.getOperator() == Operator.ADD
20         || next.getOperator() == Operator.SUBTRACT) {
21         result = applyOp(result, processing.getOperator(), processing.getNumber());
22         processing = null;
23     }
24 }
25
26 return result;
27 }
28
29 /* Collapse two terms together using the operator in secondary and the numbers
30  * from each. */
31 Term collapseTerm(Term primary, Term secondary) {
32     if (primary == null) return secondary;
33     if (secondary == null) return primary;
34
35     double value = applyOp(primary.getNumber(), secondary.getOperator(),
36                             secondary.getNumber());
37     primary.setNumber(value);
38     return primary;
39 }
40
41 double applyOp(double left, Operator op, double right) {
42     if (op == Operator.ADD) return left + right;
43     else if (op == Operator.SUBTRACT) return left - right;
44     else if (op == Operator.MULTIPLY) return left * right;
45     else if (op == Operator.DIVIDE) return left / right;
46     else return right;
47 }
48
49 public class Term {
50     public enum Operator {
51         ADD, SUBTRACT, MULTIPLY, DIVIDE, BLANK
52     }
53
54     private double value;
55     private Operator operator = Operator.BLANK;
56
57     public Term(double v, Operator op) {
58         value = v;
59         operator = op;
60     }
61
62     public double getNumber() { return value; }
63     public Operator getOperator() { return operator; }
64     public void setNumber(double v) { value = v; }
65
66     /* Parses arithmetic sequence into a list of Terms. For example, 3-5*6 becomes

```

```

67     * something like: [{BLANK,3}, {SUBTRACT, 5}, {MULTIPLY, 6}].
68     * If improperly formatted, returns null. */
69     public static ArrayList<Term> parseTermSequence(String sequence) {
70         /* Code can be found in downloadable solutions. */
71     }
72 }

```

This takes $O(N)$ time, where N is the length of the initial string.

Solution #2

Alternatively, we can solve this problem using two stacks: one for numbers and one for operators.

2 - 6 - 7 * 8 / 2 + 5

The processing works as follows:

- Each time we see a number, it gets pushed onto `numberStack`.
- Operators get pushed onto `operatorStack`—as long as the operator has higher priority than the current top of the stack. If `priority(currentOperator) <= priority(operatorStack.top())`, then we “collapse” the top of the stacks:
 - » Collapsing: pop two elements off `numberStack`, pop an operator off `operatorStack`, apply the operator, and push the result onto `numberStack`.
 - » Priority: addition and subtraction have equal priority, which is lower than the priority of multiplication and division (also equal priority).

This collapsing continues until the above inequality is broken, at which point `currentOperator` is pushed onto `operatorStack`.

- At the very end, we collapse the stack.

Let’s see this with an example: 2 - 6 - 7 * 8 / 2 + 5

	action	numberStack	operatorStack
2	<code>numberStack.push(2)</code>	2	[empty]
-	<code>operatorStack.push(-)</code>	2	-
6	<code>numberStack.push(6)</code>	6, 2	-
-	<code>collapseStacks [2 - 6]</code> <code>operatorStack.push(-)</code>	-4 -4	[empty] -
7	<code>numberStack.push(7)</code>	7, -4	=
*	<code>operatorStack.push(*)</code>	7, -4	*, -
8	<code>numberStack.push(8)</code>	8, 7, -4	*, -
/	<code>collapseStack [7 * 8]</code> <code>numberStack.push(/)</code>	56, -4 56, -4	- /, -
2	<code>numberStack.push(2)</code>	2, 56, -4	/, -
+	<code>collapseStack [56 / 2]</code> <code>collapseStack [-4 - 28]</code> <code>operatorStack.push(+)</code>	28, -4 -32 -32	- [empty] +
5	<code>numberStack.push(5)</code>	5, -32	+
	<code>collapseStack [-32 + 5]</code>	-27	[empty]
	<code>return -27</code>		

The code below implements this algorithm.

```

1  public enum Operator {
2      ADD, SUBTRACT, MULTIPLY, DIVIDE, BLANK
3  }
4
5  double compute(String sequence) {
6      Stack<Double> numberStack = new Stack<Double>();
7      Stack<Operator> operatorStack = new Stack<Operator>();
8
9      for (int i = 0; i < sequence.length(); i++) {
10         try {
11             /* Get number and push. */
12             int value = parseNextNumber(sequence, i);
13             numberStack.push((double) value);
14
15             /* Move to the operator. */
16             i += Integer.toString(value).length();
17             if (i >= sequence.length()) {
18                 break;
19             }
20
21             /* Get operator, collapse top as needed, push operator. */
22             Operator op = parseNextOperator(sequence, i);
23             collapseTop(op, numberStack, operatorStack);
24             operatorStack.push(op);
25         } catch (NumberFormatException ex) {
26             return Integer.MIN_VALUE;
27         }
28     }
29
30     /* Do final collapse. */
31     collapseTop(Operator.BLANK, numberStack, operatorStack);
32     if (numberStack.size() == 1 && operatorStack.size() == 0) {
33         return numberStack.pop();
34     }
35     return 0;
36 }
37
38 /* Collapse top until priority(futureTop) > priority(top). Collapsing means to pop
39 * the top 2 numbers and apply the operator popped from the top of the operator
40 * stack, and then push that onto the numbers stack.*/
41 void collapseTop(Operator futureTop, Stack<Double> numberStack,
42                 Stack<Operator> operatorStack) {
43     while (operatorStack.size() >= 1 && numberStack.size() >= 2) {
44         if (priorityOfOperator(futureTop) <=
45             priorityOfOperator(operatorStack.peek())) {
46             double second = numberStack.pop();
47             double first = numberStack.pop();
48             Operator op = operatorStack.pop();
49             double collapsed = applyOp(first, op, second);
50             numberStack.push(collapsed);
51         } else {
52             break;
53         }
54     }

```

```

55 }
56
57 /* Return priority of operator. Mapped so that:
58 *      addition == subtraction < multiplication == division. */
59 int priorityOfOperator(Operator op) {
60     switch (op) {
61         case ADD: return 1;
62         case SUBTRACT: return 1;
63         case MULTIPLY: return 2;
64         case DIVIDE: return 2;
65         case BLANK: return 0;
66     }
67     return 0;
68 }
69
70 /* Apply operator: left [op] right. */
71 double applyOp(double left, Operator op, double right) {
72     if (op == Operator.ADD) return left + right;
73     else if (op == Operator.SUBTRACT) return left - right;
74     else if (op == Operator.MULTIPLY) return left * right;
75     else if (op == Operator.DIVIDE) return left / right;
76     else return right;
77 }
78
79 /* Return the number that starts at offset. */
80 int parseNextNumber(String seq, int offset) {
81     StringBuilder sb = new StringBuilder();
82     while (offset < seq.length() && Character.isDigit(seq.charAt(offset))) {
83         sb.append(seq.charAt(offset));
84         offset++;
85     }
86     return Integer.parseInt(sb.toString());
87 }
88
89 /* Return the operator that occurs as offset. */
90 Operator parseNextOperator(String sequence, int offset) {
91     if (offset < sequence.length()) {
92         char op = sequence.charAt(offset);
93         switch (op) {
94             case '+': return Operator.ADD;
95             case '-': return Operator.SUBTRACT;
96             case '*': return Operator.MULTIPLY;
97             case '/': return Operator.DIVIDE;
98         }
99     }
100     return Operator.BLANK;
101 }

```

This code also takes $O(N)$ time, where N is the length of the string.

This solution involves a lot of annoying string parsing code. Remember that getting all these details out is not that important in an interview. In fact, your interviewer might even let you assume the expression is passed in pre-parsed into some sort of data structure.

Focus on modularizing your code from the beginning and “farming out” tedious or less interesting parts of the code to other functions. You want to focus on getting the core `compute` function working. The rest of the details can wait!

17

Solutions to Hard

17.1 Add Without Plus: Write a function that adds two numbers. You should not use + or any arithmetic operators.

pg 186

SOLUTION

Our first instinct in problems like these should be that we're going to have to work with bits. Why? Because when you take away the + sign, what other choice do we have? Plus, that's how computers do it!

Our next thought should be to deeply understand how addition works. We can walk through an addition problem to see if we can understand something new—some pattern—and then see if we can replicate that with code.

So let's do just that—let's walk through an addition problem. We'll work in base 10 so that it's easier to see.

To add $759 + 674$, I would usually add `digit[0]` from each number, carry the one, add `digit[1]` from each number, carry the one, and so on. You could take the same approach in binary: add each digit, and carry the one as necessary.

Can we make this a little easier? Yes! Imagine I decided to split apart the "addition" and "carry" steps. That is, I do the following:

1. Add $759 + 674$, but "forget" to carry. I then get 323.
2. Add $759 + 674$ but only do the carrying, rather than the addition of each digit. I then get 1110.
3. Add the result of the first two operations (recursively, using the same process described in step 1 and 2):
 $1110 + 323 = 1433$.

Now, how would we do this in binary?

1. If I add two binary numbers together, but forget to carry, the i th bit in the sum will be 0 only if a and b have the same i th bit (both 0 or both 1). This is essentially an XOR.
2. If I add two numbers together but *only* carry, I will have a 1 in the i th bit of the sum only if bits $i - 1$ of a and b are both 1s. This is an AND, shifted.
3. Now, recurse until there's nothing to carry.

The following code implements this algorithm.

```
1 int add(int a, int b) {
2     if (b == 0) return a;
3     int sum = a ^ b; // add without carrying
4     int carry = (a & b) << 1; // carry, but don't add
```

```

5     return add(sum, carry); // recurse with sum + carry
6 }

```

Alternatively, you can implement this iteratively.

```

1 int add(int a, int b) {
2     while (b != 0) {
3         int sum = a ^ b; // add without carrying
4         int carry = (a & b) << 1; // carry, but don't add
5         a = sum;
6         b = carry;
7     }
8     return a;
9 }

```

Problems requiring us to implement core operations like addition and subtraction are relatively common. The key in all of these problems is to dig into how these operations are usually implemented, so that we can re-implement them with the constraints of the given problem.

17.2 Shuffle: Write a method to shuffle a deck of cards. It must be a perfect shuffle—in other words, each of the $52!$ permutations of the deck has to be equally likely. Assume that you are given a random number generator which is perfect.

pg 186

SOLUTION

This is a very well known interview question, and a well known algorithm. If you aren't one of the lucky few to already know this algorithm, read on.

Let's imagine our n -element array. Suppose it looks like this:

```
[1] [2] [3] [4] [5]
```

Using our Base Case and Build approach, we can ask this question: suppose we had a method `shuffle(...)` that worked on $n - 1$ elements. Could we use this to shuffle n elements?

Sure. In fact, that's quite easy. We would first shuffle the first $n - 1$ elements. Then, we would take the n th element and randomly swap it with an element in the array. That's it!

Recursively, that algorithm looks like this:

```

1  /* Random number between lower and higher, inclusive */
2  int rand(int lower, int higher) {
3      return lower + (int)(Math.random() * (higher - lower + 1));
4  }
5
6  int[] shuffleArrayRecursively(int[] cards, int i) {
7      if (i == 0) return cards;
8
9      shuffleArrayRecursively(cards, i - 1); // Shuffle earlier part
10     int k = rand(0, i); // Pick random index to swap with
11
12     /* Swap element k and i */
13     int temp = cards[k];
14     cards[k] = cards[i];
15     cards[i] = temp;
16
17     /* Return shuffled array */
18     return cards;

```

```
19 }
```

What would this algorithm look like iteratively? Let's think about it. All it does is moving through the array and, for each element i , swapping `array[i]` with a random element between 0 and i , inclusive.

This is actually a very clean algorithm to implement iteratively:

```
1 void shuffleArrayIteratively(int[] cards) {
2     for (int i = 0; i < cards.length; i++) {
3         int k = rand(0, i);
4         int temp = cards[k];
5         cards[k] = cards[i];
6         cards[i] = temp;
7     }
8 }
```

The iterative approach is usually how we see this algorithm written.

17.3 Random Set: Write a method to randomly generate a set of m integers from an array of size n . Each element must have equal probability of being chosen.

pg 186

SOLUTION

Like the prior problem which was similar, (problem 17.2 on page 531), we can look at this problem recursively using the Base Case and Build approach.

Suppose we have an algorithm that can pull a random set of m elements from an array of size $n - 1$. How can we use this algorithm to pull a random set of m elements from an array of size n ?

We can first pull a random set of size m from the first $n - 1$ elements. Then, we just need to decide if `array[n]` should be inserted into our subset (which would require pulling out a random element from it). An easy way to do this is to pick a random number k from 0 through n . If $k < m$, then insert `array[n]` into `subset[k]`. This will both "fairly" (i.e., with proportional probability) insert `array[n]` into the subset and "fairly" remove a random element from the subset.

The pseudocode for this recursive algorithm would look like this:

```
1 int[] pickMRecursively(int[] original, int m, int i) {
2     if (i + 1 == m) { // Base case
3         /* return first m elements of original */
4     } else if (i + 1 > m) {
5         int[] subset = pickMRecursively(original, m, i - 1);
6         int k = random value between 0 and i, inclusive
7         if (k < m) {
8             subset[k] = original[i];
9         }
10        return subset;
11    }
12    return null;
13 }
```

This is even cleaner to write iteratively. In this approach, we initialize an array `subset` to be the first m elements in `original`. Then, we iterate through the array, starting at element m , inserting `array[i]` into the subset at (random) position k whenever $k < m$.

```
1 int[] pickMIteratively(int[] original, int m) {
2     int[] subset = new int[m];
3 }
```

```

4  /* Fill in subset array with first part of original array */
5  for (int i = 0; i < m ; i++) {
6      subset[i] = original[i];
7  }
8
9  /* Go through rest of original array. */
10 for (int i = m; i < original.length; i++) {
11     int k = rand(0, i); // Random # between 0 and i, inclusive
12     if (k < m) {
13         subset[k] = original[i];
14     }
15 }
16
17 return subset;
18 }

```

Both solutions are, not surprisingly, very similar to the algorithm to shuffle an array.

17.4 Missing Number: An array *A* contains all the integers from 0 to *n*, except for one number which is missing. In this problem, we cannot access an entire integer in *A* with a single operation. The elements of *A* are represented in binary, and the only operation we can use to access them is “fetch the *j*th bit of *A*[*i*],” which takes constant time. Write code to find the missing integer. Can you do it in $O(n)$ time?

pg 186

SOLUTION

You may have seen a very similar sounding problem: Given a list of numbers from 0 to *n*, with exactly one number removed, find the missing number. This problem can be solved by simply adding the list of numbers and comparing it to the actual sum of 0 through *n*, which is $\frac{n(n+1)}{2}$. The difference will be the missing number.

We could solve this by computing the value of each number, based on its binary representation, and calculating the sum.

The runtime of this solution is $n * \text{length}(n)$, when length is the number of bits in *n*. Note that $\text{length}(n) = \log_2(n)$. So, the runtime is actually $O(n \log(n))$. Not quite good enough!

So how else can we approach it?

We can actually use a similar approach, but leverage the bit values more directly.

Picture a list of binary numbers (the ----- indicates the value that was removed):

00000	00100	01000	01100
00001	00101	01001	01101
00010	00110	01010	
-----	00111	01011	

Removing the number above creates an imbalance of 1s and 0s in the least significant bit, which we’ll call *LSB*. In a list of numbers from 0 to *n*, we would expect there to be the same number of 0s as 1s (if *n* is odd), or an additional 0 if *n* is even. That is:

```

if n % 2 == 1 then count(0s) = count(1s)
if n % 2 == 0 then count(0s) = 1 + count(1s)

```

Note that this means that $\text{count}(0s)$ is always greater than or equal to $\text{count}(1s)$.

When we remove a value v from the list, we'll know immediately if v is even or odd just by looking at the least significant bits of all the other values in the list.

	$n \% 2 == 0$ $\text{count}(0s) = 1 + \text{count}(1s)$	$n \% 2 == 1$ $\text{count}(0s) = \text{count}(1s)$
$v \% 2 == 0$ $\text{LSB}_1(v) = 0$	a 0 is removed. $\text{count}(0s) = \text{count}(1s)$	a 0 is removed. $\text{count}(0s) < \text{count}(1s)$
$v \% 2 == 1$ $\text{LSB}_1(v) = 1$	a 1 is removed. $\text{count}(0s) > \text{count}(1s)$	a 1 is removed. $\text{count}(0s) > \text{count}(1s)$

So, if $\text{count}(0s) \leq \text{count}(1s)$, then v is even. If $\text{count}(0s) > \text{count}(1s)$, then v is odd.

We can now remove all the evens and focus on the odds, or remove all the odds and focus on the evens.

Okay, but how do we figure out what the next bit in v is? If v were contained in our (now smaller) list, then we should expect to find the following (where count_2 indicates the number of 0s or 1s in the second least significant bit):

$$\text{count}_2(0s) = \text{count}_2(1s) \quad \text{OR} \quad \text{count}_2(0s) = 1 + \text{count}_2(1s)$$

As in the earlier example, we can deduce the value of the second least significant bit (LSB_2) of v .

	$\text{count}_2(0s) = 1 + \text{count}_2(1s)$	$\text{count}_2(0s) = \text{count}_2(1s)$
$\text{LSB}_2(v) == 0$	a 0 is removed. $\text{count}_2(0s) = \text{count}_2(1s)$	a 0 is removed. $\text{count}_2(0s) < \text{count}_2(1s)$
$\text{LSB}_2(v) == 1$	a 1 is removed. $\text{count}_2(0s) > \text{count}_2(1s)$	a 1 is removed. $\text{count}_2(0s) > \text{count}_2(1s)$

Again, we have the same conclusion:

- If $\text{count}_2(0s) \leq \text{count}_2(1s)$, then $\text{LSB}_2(v) = 0$.
- If $\text{count}_2(0s) > \text{count}_2(1s)$, then $\text{LSB}_2(v) = 1$.

We can repeat this process for each bit. On each iteration, we count the number of 0s and 1s in bit i to check if $\text{LSB}_i(v)$ is 0 or 1. Then, we discard the numbers where $\text{LSB}_i(x) \neq \text{LSB}_i(v)$. That is, if v is even, we discard the odd numbers, and so on.

By the end of this process, we will have computed all bits in v . In each successive iteration, we look at n , then $n / 2$, then $n / 4$, and so on, bits. This results in a runtime of $O(N)$.

If it helps, we can also move through this more visually. In the first iteration, we start with all the numbers:

```

00000      00100      01000      01100
00001      00101      01001      01101
00010      00110      01010
-----      00111      01011

```

Since $\text{count}_1(0s) > \text{count}_1(1s)$, we know that $\text{LSB}_1(v) = 1$. Now, discard all numbers x where $\text{LSB}_1(x) \neq \text{LSB}_1(v)$.

```

00000      00100      01000      01100
00001      00101      01001      01101
00010      00110      01010
-----      00111      01011

```

Now, $\text{count}_2(0s) > \text{count}_2(1s)$, so we know that $\text{LSB}_2(v) = 1$. Now, discard all numbers x where $\text{LSB}_2(x) \neq \text{LSB}_2(v)$.

00000	00100	01000	01100
00001	00101	01001	01101
00010	00110	01010	
-----	00111	01011	

This time, $\text{count}_3(0s) \leq \text{count}_3(1s)$, we know that $\text{LSB}_3(v) = 0$. Now, discard all numbers x where $\text{LSB}_3(x) \neq \text{LSB}_3(v)$.

00000	00100	01000	01100
00001	00101	01001	01101
00010	00110	01010	
-----	00111	01011	

We're down to just one number. In this case, $\text{count}_4(0s) \leq \text{count}_4(1s)$, so $\text{LSB}_4(v) = 0$.

When we discard all numbers where $\text{LSB}_4(x) \neq 0$, we'll wind up with an empty list. Once the list is empty, then $\text{count}_1(0s) \leq \text{count}_1(1s)$, so $\text{LSB}_i(v) = 0$. In other words, once we have an empty list, we can fill in the rest of the bits of v with 0.

This process will compute that, for the example above, $v = 00011$.

The code below implements this algorithm. We've implemented the discarding aspect by partitioning the array by bit value as we go.

```

1  int findMissing(ArrayList<BitInteger> array) {
2      /* Start from the least significant bit, and work our way up */
3      return findMissing(array, 0);
4  }
5
6  int findMissing(ArrayList<BitInteger> input, int column) {
7      if (column >= BitInteger.INTEGER_SIZE) { // We're done!
8          return 0;
9      }
10     ArrayList<BitInteger> oneBits = new ArrayList<BitInteger>(input.size()/2);
11     ArrayList<BitInteger> zeroBits = new ArrayList<BitInteger>(input.size()/2);
12
13     for (BitInteger t : input) {
14         if (t.fetch(column) == 0) {
15             zeroBits.add(t);
16         } else {
17             oneBits.add(t);
18         }
19     }
20     if (zeroBits.size() <= oneBits.size()) {
21         int v = findMissing(zeroBits, column + 1);
22         return (v << 1) | 0;
23     } else {
24         int v = findMissing(oneBits, column + 1);
25         return (v << 1) | 1;
26     }
27 }

```

In lines 24 and 27, we recursively calculate the other bits of v . Then, we insert either a 0 or 1, depending on whether or not $\text{count}_1(0s) \leq \text{count}_1(1s)$.

17.5 Letters and Numbers: Given an array filled with letters and numbers, find the longest subarray with an equal number of letters and numbers.

pg 186

SOLUTION

In the introduction, we discussed the importance of creating a really good, general-purpose example. That's absolutely true. It's also important, though, to understand what matters.

In this case, we just want an equal number of letters and numbers. All letters are treated identically and all numbers are treated identically. Therefore, we can use an example with a single letter and a single number—or, for that matter, As and Bs, Os and 1s, or Thing1s and Thing2s.

With that said, let's start with an example:

[A, B, A, A, A, B, B, B, A, B, A, A, B, B, A, A, A, A, A]

We're looking for the smallest subarray where `count(A, subarray) = count(B, subarray)`.

Brute Force

Let's start with the obvious solution. Just go through all subarrays, count the number of As and Bs (or letters and numbers), and find the longest one that is equal.

We can make one small optimization to this. We can start with the longest subarray and, as soon as we find one which fits this equality condition, return it.

```

1  /* Return the largest subarray with equal number of 0s and 1s. Look at each
2  * subarray, starting from the longest. As soon as we find one that's equal, we
3  * return.
4  char[] findLongestSubarray(char[] array) {
5      for (int len = array.length; len > 1; len--) {
6          for (int i = 0; i <= array.length - len; i++) {
7              if (hasEqualLettersNumbers(array, i, i + len - 1)) {
8                  return extractSubarray(array, i, i + len - 1);
9              }
10         }
11     }
12     return null;
13 }
14
15 /* Check if subarray has equal number of letters and numbers. */
16 boolean hasEqualLettersNumbers(char[] array, int start, int end) {
17     int counter = 0;
18     for (int i = start; i <= end; i++) {
19         if (Character.isLetter(array[i])) {
20             counter++;
21         } else if (Character.isDigit(array[i])) {
22             counter--;
23         }
24     }
25     return counter == 0;
26 }
27
28 /* Return subarray of array between start and end (inclusive). */
29 char[] extractSubarray(char[] array, int start, int end) {
30     char[] subarray = new char[end - start + 1];
31     for (int i = start; i <= end; i++) {
32         subarray[i - start] = array[i];

```

```

33     }
34     return subarray;
35 }

```

Despite the one optimization we made, this algorithm is still $O(N^2)$, where N is the length of the array.

Optimal Solution

What we're trying to do is find a subarray where the count of letters equals the count of numbers. What if we just started from the beginning, counting the number of letters and numbers?

	a	a	a	a	1	1	a	1	1	a	a	1	a	a	1	a	a	a	a	a
#a	1	2	3	4	4	4	5	5	5	6	7	7	8	9	9	10	11	12	13	14
#1	0	0	0	0	1	2	2	3	4	4	4	5	5	5	6	6	6	6	6	6

Certainly, whenever the number of letters equals the number of numbers, we can say that from index 0 to that index is an "equal" subarray.

That will only tell us equal subarrays that start at index 0. How can we identify all equal subarrays?

Let's picture this. Suppose we inserted an equal subarray (like a11a1a) after an array like a1aaa1. How would that impact the counts?

	a	1	a	a	a	1		a	1	1	a	1	a
#a	1	1	2	3	4	4		5	5	5	6	6	7
#1	0	1	1	1	1	2		2	3	4	4	5	5

Study the numbers before the subarray (4, 2) and the end (7, 5). You might notice that, while the values aren't the same, the differences are: $4 - 2 = 7 - 5$. This makes sense. Since they've added the same number of letters and numbers, they should maintain the same difference.

Observe that when the difference is the same, the subarray starts one after the initial matching index and continues through the final matching index. This explains line 10 in the code below.

Let's update the earlier array with the differences.

	a	a	a	a	1	1	a	1	1	a	a	1	a	a	1	a	a	a	a	a
#a	1	2	3	4	4	4	5	5	5	6	7	7	8	9	9	10	11	12	13	14
#1	0	0	0	0	1	2	2	3	4	4	4	5	5	5	6	6	6	6	6	6
-	1	2	3	4	3	2	3	2	1	2	3	2	3	4	3	4	5	6	7	8

Whenever we return the same difference, then we know we have found an equal subarray. To find the biggest subarray, we just have to find the two indices farthest apart with the same value.

To do so, we use a hash table to store the first time we see a particular difference. Then, each time we see the same difference, we see if this subarray (from first occurrence of this index to current index) is bigger than the current max. If so, we update the max.

```

1  char[] findLongestSubarray(char[] array) {
2      /* Compute deltas between count of numbers and count of letters. */
3      int[] deltas = computeDeltaArray(array);
4
5      /* Find pair in deltas with matching values and largest span. */
6      int[] match = findLongestMatch(deltas);
7
8      /* Return the subarray. Note that it starts one *after* the initial occurrence of
9       * this delta. */
10     return extract(array, match[0] + 1, match[1]);
11 }
12

```

```

13 /* Compute the difference between the number of letters and numbers between the
14  * beginning of the array and each index. */
15 int[] computeDeltaArray(char[] array) {
16     int[] deltas = new int[array.length];
17     int delta = 0;
18     for (int i = 0; i < array.length; i++) {
19         if (Character.isLetter(array[i])) {
20             delta++;
21         } else if (Character.isDigit(array[i])) {
22             delta--;
23         }
24         deltas[i] = delta;
25     }
26     return deltas;
27 }
28
29 /* Find the matching pair of values in the deltas array with the largest
30  * difference in indices. */
31 int[] findLongestMatch(int[] deltas) {
32     HashMap<Integer, Integer> map = new HashMap<Integer, Integer>();
33     map.put(0, -1);
34     int[] max = new int[2];
35     for (int i = 0; i < deltas.length; i++) {
36         if (!map.containsKey(deltas[i])) {
37             map.put(deltas[i], i);
38         } else {
39             int match = map.get(deltas[i]);
40             int distance = i - match;
41             int longest = max[1] - max[0];
42             if (distance > longest) {
43                 max[1] = i;
44                 max[0] = match;
45             }
46         }
47     }
48     return max;
49 }
50
51 char[] extract(char[] array, int start, int end) { /* same */ }

```

This solution takes $O(N)$ time, where N is size of the array.

17.6 Count of 2s: Write a method to count the number of 2s between 0 and n .

pg 186

SOLUTION

Our first approach to this problem can be—and probably should be—a brute force solution. Remember that interviewers want to see how you're approaching a problem. Offering a brute force solution is a great way to start.

```

1 /* Counts the number of '2' digits between 0 and n */
2 int numberOf2sInRange(int n) {
3     int count = 0;
4     for (int i = 2; i <= n; i++) { // Might as well start at 2
5         count += numberOf2s(i);

```

```

6     }
7     return count;
8 }
9
10 /* Counts the number of '2' digits in a single number */
11 int numberOf2s(int n) {
12     int count = 0;
13     while (n > 0) {
14         if (n % 10 == 2) {
15             count++;
16         }
17         n = n / 10;
18     }
19     return count;
20 }

```

The only interesting part is that it's probably cleaner to separate out `numberOf2s` into a separate method. This demonstrates an eye for code cleanliness.

Improved Solution

Rather than looking at the problem by ranges of numbers, we can look at the problem digit by digit. Picture a sequence of numbers:

```

    0  1  2  3  4  5  6  7  8  9
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
...
110 111 112 113 114 115 116 117 118 119

```

We know that roughly one tenth of the time, the last digit will be a 2 since it happens once in any sequence of ten numbers. In fact, any digit is a 2 roughly one tenth of the time.

We say "roughly" because there are (very common) boundary conditions. For example, between 1 and 100, the 10's digit is a 2 exactly $\frac{1}{10}$ th of the time. However, between 1 and 37, the 10's digit is a 2 much more than $\frac{1}{10}$ th of the time.

We can work out what exactly the ratio is by looking at the three cases individually: `digit < 2`, `digit = 2`, and `digit > 2`.

Case `digit < 2`

Consider the value `x = 61523` and `d = 3`, and observe that `x[d] = 1` (that is, the `d`th digit of `x` is 1). There are 2s at the 3rd digit in the ranges 2000 - 2999, 12000 - 12999, 22000 - 22999, 32000 - 32999, 42000 - 42999, and 52000 - 52999. We will not yet have hit the range 62000 - 62999, so there are 6000 2s total in the 3rd digit. This is the same amount as if we were just counting all the 2s in the 3rd digit between 1 and 60000.

In other words, we can round *down* to the nearest 10^{d+1} , and then divide by 10, to compute the number of 2s in the `d`th digit.

```

if x[d] < 2: count2sInRangeAtDigit(x, d) =
    let y = round down to nearest  $10^{d+1}$ 
    return y / 10

```

Case digit > 2

Now, let's look at the case where d th digit of x is greater than 2 ($x[d] > 2$). We can apply almost the exact same logic to see that there are the same number of 2s in the 3rd digit in the range $0 - 63525$ as there as in the range $0 - 70000$. So, rather than rounding down, we round up.

```
if x[d] > 2: count2sInRangeAtDigit(x, d) =
    let y = round up to nearest  $10^{d+1}$ 
    return y / 10
```

Case digit = 2

The final case may be the trickiest, but it follows from the earlier logic. Consider $x = 62523$ and $d = 3$. We know that there are the same ranges of 2s from before (that is, the ranges $2000 - 2999, 12000 - 12999, \dots, 52000 - 52999$). How many appear in the 3rd digit in the final, partial range from $62000 - 62523$? Well, that should be pretty easy. It's just 524 ($62000, 62001, \dots, 62523$).

```
if x[d] = 2: count2sInRangeAtDigit(x, d) =
    let y = round down to nearest  $10^{d+1}$ 
    let z = right side of x (i.e.,  $x \% 10^d$ )
    return y / 10 + z + 1
```

Now, all you need is to iterate through each digit in the number. Implementing this code is reasonably straightforward.

```
1  int count2sInRangeAtDigit(int number, int d) {
2      int powerOf10 = (int) Math.pow(10, d);
3      int nextPowerOf10 = powerOf10 * 10;
4      int right = number % powerOf10;
5
6      int roundDown = number - number % nextPowerOf10;
7      int roundUp = roundDown + nextPowerOf10;
8
9      int digit = (number / powerOf10) % 10;
10     if (digit < 2) { // if the digit in spot digit is
11         return roundDown / 10;
12     } else if (digit == 2) {
13         return roundDown / 10 + right + 1;
14     } else {
15         return roundUp / 10;
16     }
17 }
18
19 int count2sInRange(int number) {
20     int count = 0;
21     int len = String.valueOf(number).length();
22     for (int digit = 0; digit < len; digit++) {
23         count += count2sInRangeAtDigit(number, digit);
24     }
25     return count;
26 }
```

This question requires very careful testing. Make sure to generate a list of test cases, and to work through each of them.

17.7 Baby Names: Each year, the government releases a list of the 10,000 most common baby names and their frequencies (the number of babies with that name). The only problem with this is that some names have multiple spellings. For example, "John" and "Jon" are essentially the same name but would be listed separately in the list. Given two lists, one of names/frequencies and the other of pairs of equivalent names, write an algorithm to print a new list of the true frequency of each name. Note that if John and Jon are synonyms, and Jon and Johnny are synonyms, then John and Johnny are synonyms. (It is both transitive and symmetric.) In the final list, any name can be used as the "real" name.

EXAMPLE

Input:

Names: John (15), Jon (12), Chris (13), Kris (4), Christopher (19)

Synonyms: (Jon, John), (John, Johnny), (Chris, Kris), (Chris, Christopher)

Output: John (27), Kris (36)

pg 187

SOLUTION

Let's start off with a good example. We want an example with some names with multiple synonyms and some with none. Additionally, we want the synonym list to be diverse in which name is on the left side and which is on the right. For example, we wouldn't want Johnny to always be the name on the left side as we're creating the group of (John, Jonathan, Jon, and Johnny).

This list should work fairly well.

Name	Count
John	10
Jon	3
Davis	2
Kari	3
Johnny	11
Carlton	8
Carleton	2
Jonathan	9
Carrie	5

Name	Alternate
Jonathan	John
Jon	Johnny
Johnny	John
Kari	Carrie
Carleton	Carlton

The final list should be something like: John (33), Kari (8), Davis(2), Carleton (10).

Solution #1

Let's assume our baby names list is given to us as a hash table. (If not, it's easy enough to build one.)

We can start reading pairs in from the synonyms list. As we read the pair (Jonathan, John), we can merge the counts for Jonathan and John together. We'll need to remember, though, that we saw this pair, because, in the future, we could discover that Jonathan is equivalent to something else.

We can use a hash table (L1) that maps from a name to its "true" name. We'll also need to know, given a "true" name, all the names equivalent to it. This will be stored in a hash table L2. Note that L2 acts as a reverse lookup of L1.

READ (Jonathan, John)

```

L1.ADD Jonathan -> John
L2.ADD John -> Jonathan
READ (Jon, Johnny)
L1.ADD Jon -> Johnny
L2.ADD Johnny -> Jon
READ (Johnny, John)
L1.ADD Johnny -> John
L1.UPDATE Jon -> John
L2.UPDATE John -> Jonathan, Johnny, Jon

```

If we later find that John is equivalent to, say, Jonny, we'll need to look up the names in L1 and L2 and merge together all the names that are equivalent to them.

This will work, but it's unnecessarily complicated to keep track of these two lists.

Instead, we can think of these names as "equivalence classes." When we find a pair (Jonathan, John), we put these in the same set (or equivalence classes). Each name maps to its equivalence class. All items in the set map to the same instance of the set.

If we need to merge two sets, then we copy one set into the other and update the hash table to point to the new set.

```

READ (Jonathan, John)
CREATE Set1 = Jonathan, John
L1.ADD Jonathan -> Set1
L1.ADD John -> Set1
READ (Jon, Johnny)
CREATE Set2 = Jon, Johnny
L1.ADD Jon -> Set2
L1.ADD Johnny -> Set2
READ (Johnny, John)
COPY Set2 into Set1.
    Set1 = Jonathan, John, Jon, Johnny
L1.UPDATE Jon -> Set1
L1.UPDATE Johnny -> Set1

```

In the last step above, we iterated through all items in Set2 and updated the reference to point to Set1. As we do this, we keep track of the total frequency of names.

```

1  HashMap<String, Integer> trulyMostPopular(HashMap<String, Integer> names,
2                                     String[][] synonyms) {
3      /* Parse list and initialize equivalence classes.*/
4      HashMap<String, NameSet> groups = constructGroups(names);
5
6      /* Merge equivalence classes together. */
7      mergeClasses(groups, synonyms);
8
9      /* Convert back to hash map. */
10     return convertToMap(groups);
11 }
12
13 /* This is the core of the algorithm. Read through each pair. Merge their
14 * equivalence classes and update the mapping of the secondary class to point to
15 * the first set.*/
16 void mergeClasses(HashMap<String, NameSet> groups, String[][] synonyms) {
17     for (String[] entry : synonyms) {
18         String name1 = entry[0];
19         String name2 = entry[1];
20         NameSet set1 = groups.get(name1);

```

```

21     NameSet set2 = groups.get(name2);
22     if (set1 != set2) {
23         /* Always merge the smaller set into the bigger one. */
24         NameSet smaller = set2.size() < set1.size() ? set2 : set1;
25         NameSet bigger = set2.size() < set1.size() ? set1 : set2;
26
27         /* Merge lists */
28         Set<String> otherNames = smaller.getNames();
29         int frequency = smaller.getFrequency();
30         bigger.copyNamesWithFrequency(otherNames, frequency);
31
32         /* Update mapping */
33         for (String name : otherNames) {
34             groups.put(name, bigger);
35         }
36     }
37 }
38 }
39
40 /* Read through (name, frequency) pairs and initialize a mapping of names to
41 * NameSets (equivalence classes).*/
42 HashMap<String, NameSet> constructGroups(HashMap<String, Integer> names) {
43     HashMap<String, NameSet> groups = new HashMap<String, NameSet>();
44     for (Entry<String, Integer> entry : names.entrySet()) {
45         String name = entry.getKey();
46         int frequency = entry.getValue();
47         NameSet group = new NameSet(name, frequency);
48         groups.put(name, group);
49     }
50     return groups;
51 }
52
53 HashMap<String, Integer> convertToMap(HashMap<String, NameSet> groups) {
54     HashMap<String, Integer> list = new HashMap<String, Integer>();
55     for (NameSet group : groups.values()) {
56         list.put(group.getRootName(), group.getFrequency());
57     }
58     return list;
59 }
60
61 public class NameSet {
62     private Set<String> names = new HashSet<String>();
63     private int frequency = 0;
64     private String rootName;
65
66     public NameSet(String name, int freq) {
67         names.add(name);
68         frequency = freq;
69         rootName = name;
70     }
71
72     public void copyNamesWithFrequency(Set<String> more, int freq) {
73         names.addAll(more);
74         frequency += freq;
75     }
76 }

```

```

77 public Set<String> getNames() { return names; }
78 public String getRootName() { return rootName; }
79 public int getFrequency() { return frequency; }
80 public int size() { return names.size(); }
81 }

```

The runtime of the algorithm is a bit tricky to figure out. One way to think about it is to think about what the worst case is.

For this algorithm, the worst case is where all names are equivalent—and we have to constantly merge sets together. Also, for the worst case, the merging should come in the worst possible way: repeated pairwise merging of sets. Each merging requires copying the set's elements into an existing set and updating the pointers from those items. It's slowest when the sets are larger.

If you notice the parallel with merge sort (where you have to merge single-element arrays into two-element arrays, and then two-element arrays into four-element arrays, until finally having a full array), you might guess it's $O(N \log N)$. That is correct.

If you don't notice that parallel, here's another way to think about it.

Imagine we had the names (a, b, c, d, . . . , z). In our worst case, we'd first pair up the items into equivalence classes: (a, b), (c, d), (e, f), . . . , (y, z). Then, we'd merge pairs of those: (a, b, c, d), (e, f, g, h), . . . , (w, x, y, z). We'd continue doing this until we wind up with just one class.

At each "sweep" through the list where we merge sets together, half of the items get moved into a new set. This takes $O(N)$ work per sweep. (There are fewer sets to merge, but each set has grown larger.)

How many sweeps do we do? At each sweep, we have half as many sets as we did before. Therefore, we do $O(\log N)$ sweeps.

Since we're doing $O(\log N)$ sweeps and $O(N)$ work per sweep, the total runtime is $O(N \log N)$.

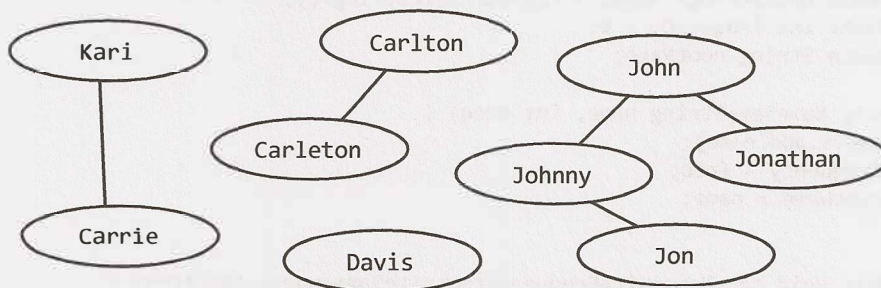
This is pretty good, but let's see if we can make it even faster.

Optimized Solution

To optimize the old solution, we should think about what exactly makes it slow. Essentially, it's the merging and updating of pointers.

So what if we just didn't do that? What if we marked that there was an equivalence relationship between two names, but didn't actually do anything with the information yet?

In this case, we'd be building essentially a graph.



Now what? Visually, it seems easy enough. Each component is an equivalent set of names. We just need to group the names by their component, sum up their frequencies, and return a list with one arbitrarily chosen name from each group.

In practice, how does this work? We could pick a name and do a depth-first (or breadth-first) search to sum the frequencies of all the names in one component. We would have to make sure that we hit each component exactly once. That's easy enough to achieve: mark a node as `visited` after it's discovered in the graph search, and only start the search for nodes where `visited` is false.

```

1  HashMap<String, Integer> trulyMostPopular(HashMap<String, Integer> names,
2                                          String[][] synonyms) {
3      /* Create data. */
4      Graph graph = constructGraph(names);
5      connectEdges(graph, synonyms);
6
7      /* Find components. */
8      HashMap<String, Integer> rootNames = getTrueFrequencies(graph);
9      return rootNames;
10 }
11
12 /* Add all names to graph as nodes. */
13 Graph constructGraph(HashMap<String, Integer> names) {
14     Graph graph = new Graph();
15     for (Entry<String, Integer> entry : names.entrySet()) {
16         String name = entry.getKey();
17         int frequency = entry.getValue();
18         graph.createNode(name, frequency);
19     }
20     return graph;
21 }
22
23 /* Connect synonymous spellings. */
24 void connectEdges(Graph graph, String[][] synonyms) {
25     for (String[] entry : synonyms) {
26         String name1 = entry[0];
27         String name2 = entry[1];
28         graph.addEdge(name1, name2);
29     }
30 }
31
32 /* Do DFS of each component. If a node has been visited before, then its component
33  * has already been computed. */
34 HashMap<String, Integer> getTrueFrequencies(Graph graph) {
35     HashMap<String, Integer> rootNames = new HashMap<String, Integer>();
36     for (GraphNode node : graph.getNodes()) {
37         if (!node.isVisited()) { // Already visited this component
38             int frequency = getComponentFrequency(node);
39             String name = node.getName();
40             rootNames.put(name, frequency);
41         }
42     }
43     return rootNames;
44 }
45
46 /* Do depth-first search to find the total frequency of this component, and mark
47  * each node as visited.*/
48 int getComponentFrequency(GraphNode node) {
49     if (node.isVisited()) return 0; // Already visited
50
51     node.setIsVisited(true);
52     int sum = node.getFrequency();

```

```
53     for (GraphNode child : node.getNeighbors()) {
54         sum += getComponentFrequency(child);
55     }
56     return sum;
57 }
58
59 /* Code for GraphNode and Graph is fairly self-explanatory, but can be found in
60 * the downloadable code solutions.*/
```

To analyze the efficiency, we can think about the efficiency of each part of the algorithm.

- Reading in the data is linear with respect to the size of the data, so it takes $O(B + P)$ time, where B is the number of baby names and P is the number of pairs of synonyms. This is because we only do a constant amount of work per piece of input data.
- To compute the frequencies, each edge gets “touched” exactly once across all of the graph searches and each node gets touched exactly once to check if it’s been visited. The time of this part is $O(B + P)$.

Therefore, the total time of the algorithm is $O(B + P)$. We know we cannot do better than this since we must at least read in the $B + P$ pieces of data.

17.8 Circus Tower: A circus is designing a tower routine consisting of people standing atop one another’s shoulders. For practical and aesthetic reasons, each person must be both shorter and lighter than the person below him or her. Given the heights and weights of each person in the circus, write a method to compute the largest possible number of people in such a tower.

pg 187

SOLUTION

When we cut out all the “fluff” to this problem, we can understand that the problem is really the following.

We have a list of pairs of items. Find the longest sequence such that both the first and second items are in non-decreasing order.

One thing we might first try is sorting the items on an attribute. This is useful actually, but it won’t get us all the way there.

By sorting the items by height, we have a relative order the items must appear in. We still need to find the longest increasing subsequence of weight though.

Solution 1: Recursive

One approach is to essentially try all possibilities. After sorting by height, we iterate through the array. At each element, we branch into two choices: add this element to the subsequence (if it’s valid) or do not.

```
1  ArrayList<HtWt> longestIncreasingSeq(ArrayList<HtWt> items) {
2      Collections.sort(items);
3      return bestSeqAtIndex(items, new ArrayList<HtWt>(), 0);
4  }
5
6  ArrayList<HtWt> bestSeqAtIndex(ArrayList<HtWt> array, ArrayList<HtWt> sequence,
7                                int index) {
8      if (index >= array.size()) return sequence;
9
10     HtWt value = array.get(index);
11
```

```

12     ArrayList<HtWt> bestWith = null;
13     if (canAppend(sequence, value)) {
14         ArrayList<HtWt> sequenceWith = (ArrayList<HtWt>) sequence.clone();
15         sequenceWith.add(value);
16         bestWith = bestSeqAtIndex(array, sequenceWith, index + 1);
17     }
18
19     ArrayList<HtWt> bestWithout = bestSeqAtIndex(array, sequence, index + 1);
20
21     if (bestWith == null || bestWithout.size() > bestWith.size()) {
22         return bestWithout;
23     } else {
24         return bestWith;
25     }
26 }
27
28 boolean canAppend(ArrayList<HtWt> solution, HtWt value) {
29     if (solution == null) return false;
30     if (solution.size() == 0) return true;
31
32     HtWt last = solution.get(solution.size() - 1);
33     return last.isBefore(value);
34 }
35
36 ArrayList<HtWt> max(ArrayList<HtWt> seq1, ArrayList<HtWt> seq2) {
37     if (seq1 == null) {
38         return seq2;
39     } else if (seq2 == null) {
40         return seq1;
41     }
42     return seq1.size() > seq2.size() ? seq1 : seq2;
43 }
44
45 public class HtWt implements Comparable<HtWt> {
46     private int height;
47     private int weight;
48     public HtWt(int h, int w) { height = h; weight = w; }
49
50     public int compareTo(HtWt second) {
51         if (this.height != second.height) {
52             return ((Integer)this.height).compareTo(second.height);
53         } else {
54             return ((Integer)this.weight).compareTo(second.weight);
55         }
56     }
57
58     /* Returns true if "this" should be lined up before "other". Note that it's
59      * possible that this.isBefore(other) and other.isBefore(this) are both false.
60      * This is different from the compareTo method, where if a < b then b > a. */
61     public boolean isBefore(HtWt other) {
62         if (height < other.height && weight < other.weight) {
63             return true;
64         } else {
65             return false;
66         }
67     }

```

```
68 }
```

This algorithm will take $O(2^n)$ time. We can optimize it using memoization (that is, caching the best sequences).

There's a cleaner way to do this though.

Solution #2: Iterative

Imagine we had the longest subsequence that terminates with each element, $A[0]$ through $A[3]$. Could we use this to find the longest subsequence that terminates with $A[4]$?

```
Array: 13, 14, 10, 11, 12
Longest(ending with A[0]): 13
Longest(ending with A[1]): 13, 14
Longest(ending with A[2]): 10
Longest(ending with A[3]): 10, 11
Longest(ending with A[4]): 10, 11, 12
```

Sure. We just append $A[4]$ on to the longest subsequence that it can be appended to.

This is now fairly straightforward to implement.

```
1  ArrayList<HtWt> longestIncreasingSeq(ArrayList<HtWt> array) {
2      Collections.sort(array);
3
4      ArrayList<ArrayList<HtWt>> solutions = new ArrayList<ArrayList<HtWt>>();
5      ArrayList<HtWt> bestSequence = null;
6
7      /* Find the longest subsequence that terminates with each element. Track the
8       * longest overall subsequence as we go. */
9      for (int i = 0; i < array.size(); i++) {
10         ArrayList<HtWt> longestAtIndex = bestSeqAtIndex(array, solutions, i);
11         solutions.add(i, longestAtIndex);
12         bestSequence = max(bestSequence, longestAtIndex);
13     }
14
15     return bestSequence;
16 }
17
18 /* Find the longest subsequence which terminates with this element. */
19 ArrayList<HtWt> bestSeqAtIndex(ArrayList<HtWt> array,
20     ArrayList<ArrayList<HtWt>> solutions, int index) {
21     HtWt value = array.get(index);
22
23     ArrayList<HtWt> bestSequence = new ArrayList<HtWt>();
24
25     /* Find the longest subsequence that we can append this element to. */
26     for (int i = 0; i < index; i++) {
27         ArrayList<HtWt> solution = solutions.get(i);
28         if (canAppend(solution, value)) {
29             bestSequence = max(solution, bestSequence);
30         }
31     }
32
33     /* Append element. */
34     ArrayList<HtWt> best = (ArrayList<HtWt>) bestSequence.clone();
35     best.add(value);
36 }
```

```

37     return best;
38 }

```

This algorithm operates in $O(n^2)$ time. An $O(n \log(n))$ algorithm does exist, but it is considerably more complicated and it is highly unlikely that you would derive this in an interview—even with some help. However, if you are interested in exploring this solution, a quick internet search will turn up a number of explanations of this solution.

17.9 Kth Multiple: Design an algorithm to find the k th number such that the only prime factors are 3, 5, and 7. Note that 3, 5, and 7 do not have to be factors, but it should not have any other prime factors. For example, the first several multiples would be (in order) 1, 3, 5, 7, 9, 15, 21.

pg 187

SOLUTION

Let's first understand what this problem is asking for. It's asking for the k th smallest number that is in the form $3^a * 5^b * 7^c$. Let's start with a brute force way of finding this.

Brute Force

We know that biggest this k th number could be is $3^k * 5^k * 7^k$. So, the “stupid” way of doing this is to compute $3^a * 5^b * 7^c$ for all values of a , b , and c between 0 and k . We can throw them all into a list, sort the list, and then pick the k th smallest value.

```

1  int getKthMagicNumber(int k) {
2      ArrayList<Integer> possibilities = allPossibleKFactors(k);
3      Collections.sort(possibilities);
4      return possibilities.get(k);
5  }
6
7  ArrayList<Integer> allPossibleKFactors(int k) {
8      ArrayList<Integer> values = new ArrayList<Integer>();
9      for (int a = 0; a <= k; a++) { // loop 3
10         int powA = (int) Math.pow(3, a);
11         for (int b = 0; b <= k; b++) { // loop 5
12             int powB = (int) Math.pow(5, b);
13             for (int c = 0; c <= k; c++) { // loop 7
14                 int powC = (int) Math.pow(7, c);
15                 int value = powA * powB * powC;
16
17                 /* Check for overflow. */
18                 if (value < 0 || powA == Integer.MAX_VALUE ||
19                     powB == Integer.MAX_VALUE ||
20                     powC == Integer.MAX_VALUE) {
21                     value = Integer.MAX_VALUE;
22                 }
23                 values.add(value);
24             }
25         }
26     }
27     return values;
28 }

```